TECHNICAL REFERENCE MANUAL

Semiconductor Design Infrastructure Handbook

Deployment, Configuration, and Administration of Cadence-based RHEL workstations.

AUTHOR

Anirban Das
BTech (ECE)
Class of 2027

ADVISOR

Dr. Anshika Srivastava
Program Coordinator
Department of Electronics and Communication Engineering

Gati Shakti Vishwavidyalaya
A Central University under Ministry of Railways, Government of India
Vadodara, Gujarat 390004

Abstract

this requires further enhancement

This manual covers the procedures required to deploy, provision, operate, maintain, and recover a RHEL-based semiconductor CAD workstation environment. It is intended to serve as an operational reference for administrators responsible for preparing workstations, installing EDA tools, configuring user access, and maintaining a reliable design environment over time.

Specifically, this manual covers the following areas:

- Operating system deployment for RHEL workstations, including installation planning and baseline system configuration.
- Workstation provisioning, including package installation, network readiness, storage preparation, and post-installation setup.
- EDA installation workflows for supported semiconductor design tools and their required dependencies.
- Licensing configuration, including license server connectivity, environment variables, and validation steps.
- Shell environments for CAD users, including startup files, module or tool setup scripts, and reproducible environment initialization.
- User management tasks, including account creation, group membership, access control, and basic permission practices.
- Troubleshooting procedures for common operating system, EDA tool, license, shell, and user-access issues.
- Recovery procedures for failed installations, broken environments, workstation rebuilds, and service restoration.
- Maintenance workflows, including updates, backups, configuration audits, cleanup tasks, and periodic validation of the CAD environment.

This manual does not cover semiconductor design methodology, RTL design, verification strategy, physical design theory, circuit design, device modeling, or EDA tool usage from a design-engineering perspective. It also does not define institutional IT policy, network architecture outside the workstation environment, procurement procedures, vendor contract negotiation, or license entitlement management beyond the technical configuration needed for workstation operation.

Preface

Modern civilization is built upon silicon. From mobile communication systems and medical instrumentation to aerospace control systems, artificial intelligence accelerators, and high-performance computing platforms, nearly every contemporary technological system is ultimately dependent upon integrated circuits containing billions of transistors operating at nanometer-scale geometries. The design and verification of such devices would be computationally impossible without the development of sophisticated Electronic Design Automation (EDA) frameworks capable of transforming abstract circuit descriptions into physically manufacturable semiconductor layouts. Over the past several decades, EDA has evolved from a collection of isolated drafting and simulation utilities into a deeply interconnected computational ecosystem comprising schematic capture systems, hardware description language toolchains, simulation engines, parasitic extraction frameworks, physical verification suites, and advanced layout environments operating across distributed compute infrastructure. As semiconductor complexity increased, the underlying computing environments required to support these workflows evolved alongside them, giving rise to highly specialized workstation architectures optimized for reliability, reproducibility, performance, and long-term maintainability.

Contemporary semiconductor design environments are overwhelmingly deployed on Linux-based operating systems due to their stability, scalability, scripting flexibility, network integration capabilities, and compatibility with industry-standard EDA toolchains. Unlike general-purpose desktop operating systems, enterprise Linux distributions provide deterministic package management, robust permission models, multi-user infrastructure support, and predictable system behavior required for large-scale engineering workflows. Among these distributions, Red Hat Enterprise Linux (RHEL) and its derivatives have emerged as the de facto industry standard within both commercial semiconductor organizations and academic VLSI laboratories owing to their long-term support lifecycle, binary compatibility guarantees, enterprise-grade tooling ecosystem, and extensive vendor certification support from major EDA providers. Consequently, the deployment methodology described throughout this manual is centered around a standardized RHEL-based semiconductor CAD environment designed to support scalable, reproducible, and maintainable VLSI design workflows across multiple engineering workstations.

Contents

Part I

SYSTEM SETUP

Operating System Deployment and RHEL Workstation Initialization

01

Foundations of the EDA Environment

1.1 Introduction to EDA

Electronic Design Automation (EDA) refers to the collection of software tools, computational frameworks, and engineering workflows used in the design, simulation, verification, and physical implementation of integrated circuits and semiconductor systems. Modern semiconductor devices contain millions to billions of transistors operating at extremely small geometries, making manual design methodologies computationally impractical and operationally impossible. EDA tools therefore serve as the foundational infrastructure enabling engineers to transform conceptual circuit designs into manufacturable silicon devices.

Contemporary EDA environments encompass a wide range of specialized applications including schematic capture systems, Hardware Description Language (HDL) development environments, circuit simulators, synthesis tools, physical layout editors, parasitic extraction frameworks, timing analysis engines, and design-rule verification systems. These tools collectively support the complete semiconductor development lifecycle, from behavioral modeling and functional verification to physical layout generation and fabrication validation.

The increasing complexity of Very Large Scale Integration (VLSI) systems has made EDA indispensable within both academic and industrial semiconductor design workflows. Beyond improving productivity, EDA enables precision, scalability, repeatability, and verification accuracy that would otherwise be unattainable through conventional design methodologies. As a result, modern semiconductor engineering is fundamentally dependent upon robust EDA infrastructure and the computational environments that support it.

1.2 Why Enterprise Linux Anchors Modern EDA?

Modern EDA toolchains impose demanding requirements on the underlying operating system environment, including stability, deterministic behavior, scripting flexibility, network interoperability, multi-user access control, and long-term software compatibility. Consequently, semiconductor design organizations and research laboratories overwhelmingly deploy EDA infrastructure on Linux-based operating systems rather than general-purpose desktop platforms.

Linux provides a highly modular and administratively transparent environment capable of supporting complex engineering workflows involving distributed compute resources, centralized licensing systems, shared storage infrastructure, remote administration, and automated deployment methodologies. Native shell scripting capabilities, package management systems, process control utilities, and network tooling further enable scalable workstation administration and reproducible environment configuration across large laboratory deployments.

Among available Linux distributions, enterprise-grade platforms such as **Red Hat Enterprise Linux (RHEL)** and its derivatives have become the de facto standard within semiconductor design environments. Their widespread adoption is driven by long-term support lifecycles, predictable release stability, extensive vendor certification, and broad compatibility with commercial EDA applications. Most major semiconductor

tool vendors officially validate and support their software primarily on enterprise Linux environments, making RHEL-based systems the preferred foundation for reliable and maintainable CAD infrastructure deployment.

Accordingly, the workstation architecture described throughout this manual is built upon a standardized RHEL-based environment designed to provide operational consistency, scalability, and long-term maintainability for semiconductor design laboratories and VLSI research workflows.

1.3 Red Hat Enterprise Linux as Industrial Infrastructure

The decision to standardize a semiconductor design environment on a specific Linux distribution is not a matter of administrative preference. It is an infrastructure decision with direct consequences for EDA tool compatibility, operational stability, vendor support, and long-term maintainability. Within professional semiconductor environments, Red Hat Enterprise Linux has become a preferred foundation because it provides the stability and compatibility characteristics required by commercial EDA workflows.

Commercial EDA tools are typically distributed as precompiled binary applications that depend on stable system libraries, predictable runtime behavior, and well-defined operating system interfaces. RHEL maintains Application Binary Interface and Application Programming Interface consistency within a major release, allowing validated tool installations to remain reliable across routine system updates. This consistency is especially important for tools that depend on components such as `glibc`, `libX11`, Motif libraries, OpenSSL, shell utilities, and kernel-level behavior.

RHEL also provides long-term lifecycle stability that aligns well with semiconductor design workflows. A laboratory or production design environment may depend on a specific operating system and EDA tool combination for several years. Frequent distribution-level changes can introduce dependency drift and invalidate previously working configurations. By contrast, RHEL's enterprise release model prioritizes predictable maintenance, security updates, and controlled package evolution over rapid upstream change.

Vendor certification is another important factor. Major EDA vendors commonly certify their tools against specific RHEL major versions and closely compatible derivatives. Operating within a certified platform reduces support risk, simplifies troubleshooting, and provides a defensible baseline when vendor escalation is required. While unsupported Linux distributions may sometimes run the same tools, they introduce additional uncertainty into installation, execution, and support workflows.

The RHEL ecosystem also supports maintainable administration practices through standard tools such as `dnf`, `systemd`, `lvm`, `sssd`, and `subscription-manager`. These tools provide a well-documented administrative surface for package management, service control, storage configuration, authentication integration, and system maintenance. In institutional environments where systems must be rebuilt, audited, extended, or handed over between administrators, this consistency is a practical advantage.

Throughout the remainder of this manual, operating system procedures, configuration examples, package names, service management commands, and administrative workflows are specified for a RHEL 9-compatible environment. Unless otherwise noted, the procedures also apply to Rocky Linux 9 and AlmaLinux 9 as binary-compatible RHEL derivatives. References to *RHEL* in the operational context of this document should therefore be understood to include these derivatives unless a Red Hat subscription-specific feature is explicitly required.

1.4 Design Philosophy of the EDA Environment

The semiconductor design infrastructure described in this manual is not assembled opportunistically. It is engineered according to a defined set of operational principles that govern every decision from initial workstation deployment through long-term maintenance. Understanding these principles is necessary context for the procedures that follow, because many configuration choices that might otherwise appear arbitrary are direct expressions of the underlying philosophy.

The environment is built around **standardization**. All workstations in the laboratory are deployed from a common baseline: identical operating system version, identical package selection, identical directory structures, and identical tool installation paths. Deviation from this baseline is treated as an exception requiring justification, not a routine outcome of per-machine administration. Standardization is the prerequisite for everything else.

From standardization follows **reproducibility**. A workstation rebuilt from documented procedures should produce an environment functionally identical to the one it replaces. This property matters both operationally—when a workstation fails and must be restored—and institutionally, as the laboratory's administrative knowledge must remain embedded in its documentation rather than in the accumulated state of individual machines.

The deployment is designed to **scale**. Procedures written for a single workstation apply without modification to the full laboratory fleet. Configuration that must be individually customized per machine is a design defect. Where centralized mechanisms exist—for user authentication, license server connectivity, shared storage, or environment initialization—they are used in preference to local equivalents.

Centralized administration follows from this. User accounts, access permissions, and shell environments are managed at the infrastructure level rather than configured locally on each workstation. This reduces the administrative surface area, eliminates inconsistency across machines, and ensures that changes propagate uniformly rather than requiring repeated per-host intervention.

Underlying all of these principles is the explicit goal of **eliminating configuration drift**. In long-running laboratory environments, workstations tend to diverge from one another over time as ad hoc fixes, local experiments, and undocumented modifications accumulate. The procedures in this manual are written to resist this tendency—through documented baselines, version-controlled configuration, and periodic validation practices that detect and correct deviation before it compounds.

The result is an infrastructure in which any workstation can be administered, diagnosed, or rebuilt by any administrator with access to this document, without dependence on institutional memory or machine-specific knowledge.

1.5 Overview of Laboratory Infrastructure

1.6 Workstation Hardware Requirements and Deployment Constraints

Electronic Design Automation tools impose system resource requirements that differ substantially from those of general-purpose desktop software. A typical office productivity environment or software development workstation can operate adequately on modest storage allocations, a few gigabytes of memory, and commodity processor performance. EDA environments do not share these characteristics. The software stacks involved in a complete semiconductor design workflow are architecturally large, computationally intensive, and operationally demanding in ways that must be accounted for at the infrastructure planning stage rather than addressed reactively after deployment.

1.6.1 Storage Requirements

Modern EDA suites consist of numerous interrelated tools, each with its own binaries, libraries, technology files, process design kits, documentation sets, and supporting scripts. A complete installation image for a commercial EDA suite does not occupy gigabytes in the way that a large application suite might. It occupies an order of magnitude more. The Cadence EDA installation deployed in this laboratory environment occupied approximately **480 GB** of storage prior to the creation of any user project data, simulation output, waveform databases, or temporary working files. This figure reflects only the tool installation itself.

In practice, active design work compounds storage consumption considerably. Simulation runs generate large output datasets. Layout databases accumulate across design iterations. Waveform viewers load and cache substantial binary files during interactive sessions. Verification engines write intermediate results to disk throughout their execution. Over the operational lifetime of a workstation, the storage footprint of the design environment grows continuously. Storage planning must therefore account not only for the initial

installation image but for a realistic projection of working data volume across the deployment period.

For this reason, workstations intended for sustained EDA use should be provisioned with local storage that provides meaningful headroom beyond the installation baseline. The 480 GB installation figure should be treated as a floor, not a target. Configurations that provision only enough storage to accommodate the initial installation leave no margin for project data, OS overhead, swap allocation, or tool updates, and will require administrative intervention as the environment matures.

1.6.2 Memory Requirements

Semiconductor design workflows routinely involve multiple concurrently active applications. A typical interactive session may involve a layout editor, a schematic capture environment, a circuit simulator, a waveform viewer, and a design rule checking utility operating simultaneously within the same desktop session. Each of these applications maintains its own working set in memory, and several—particularly simulation engines and verification utilities—can expand their memory consumption substantially during active computation.

Insufficient memory allocation produces predictable and disruptive consequences in this context. Operating system memory pressure forces active application data to swap, degrading interactive responsiveness and extending simulation runtimes. In severe cases, the out-of-memory subsystem may terminate running processes, corrupting simulation state and requiring workflow restart. These outcomes are not edge cases in underpowered environments; they are routine operational hazards.

Workstations deployed for full EDA workflows should be provisioned with sufficient physical memory to support concurrent tool operation without inducing swap utilization under normal working conditions. The specific requirement varies by workflow, but configurations below 16 GB of physical memory are generally unsuitable for sustained multi-tool VLSI design sessions. Workstations supporting heavier simulation workloads or concurrent multi-user access benefit from higher memory allocations accordingly.

1.6.3 Deployment Methodology: Native Installation versus Virtualization

The manner in which RHEL is deployed on a workstation has direct consequences for the resource availability of the EDA environment. Two deployment models are relevant to this laboratory context: direct installation on dedicated hardware, and virtualized deployment within a hypervisor environment running on an existing host operating system.

In a native installation—whether on a dedicated workstation or in a dual-boot configuration alongside an existing operating system—RHEL has unrestricted access to all physical hardware resources. The full complement of CPU cores, physical memory, storage throughput, and graphics capability is available to the operating system and the EDA tools running within it. This is the preferred deployment model for sustained engineering workflows, as it eliminates the resource-sharing constraints and performance overhead inherent to virtualized environments.

Figure 1.1: screenshot of hyper-V

Virtualization, however, represents a practically useful alternative under specific deployment conditions. During the initial workshop phase of this laboratory deployment, Microsoft Hyper-V was used to provision RHEL virtual machines on workstations running existing Windows installations. This approach was selected because many of the available systems contained active Windows environments that could not be immediately repartitioned, and the operational priority was to make the EDA environment accessible within a constrained preparation timeline.

The resource tradeoffs introduced by this approach are significant and should be understood clearly. The reference deployment system carried 32 GB of physical memory. Of this, 16 GB was allocated to the RHEL virtual machine, with the remaining capacity reserved for the Windows host and the memory overhead of the Hyper-V hypervisor layer itself. Under this configuration, the EDA environment operates within a resource

envelope that is materially smaller than the physical hardware could otherwise provide. CPU scheduling is mediated by the hypervisor, storage I/O is subject to virtualization overhead, and memory allocation is fixed at provisioning time rather than dynamically available from the full physical pool.

Virtualized deployment is appropriate for introductory workshops, environment familiarization, and short-duration instructional use cases in which the full performance envelope of the EDA tools is not required. It is not the preferred model for production semiconductor design workflows. As workstation preparation schedules permit, systems initially deployed under Hyper-V should be transitioned to native RHEL installations in order to provide the EDA environment with unrestricted access to the underlying hardware resources. All procedures documented in this manual apply to both deployment models unless a specific section explicitly notes otherwise.

1.7 Summary

This chapter introduced the foundational concepts underlying modern semiconductor EDA infrastructure and established the operational context for the deployment procedures described throughout this manual. The discussion examined the role of Electronic Design Automation within contemporary VLSI workflows, the infrastructure requirements imposed by commercial EDA toolchains, and the reasons enterprise Linux environments have become the standard platform for semiconductor engineering systems.

The chapter further justified the adoption of Red Hat Enterprise Linux as the primary operating system environment for workstation deployment, emphasizing considerations such as long-term stability, vendor certification, maintainability, and compatibility with commercial EDA applications. In addition, the architectural philosophy of the laboratory environment, including standardization, reproducibility, centralized administration, and scalability, was introduced as the guiding framework for all subsequent deployment and maintenance procedures.

Finally, the chapter discussed the hardware and deployment considerations associated with EDA environments, including storage requirements, memory allocation strategies, and the operational tradeoffs between native and virtualized deployment methodologies. These concepts collectively establish the technical foundation required for the operating system installation and workstation provisioning procedures described in the following chapter.